HLD Level 1.6

Replication, Partitioning / Sharding & Consistency - C++ / Systems Interview Notes

> **Core distinction: replication copies the same data across nodes; partitioning splits different data across nodes. Replication mainly helps availability/read scale; partitioning mainly helps capacity/write scale.**

1. Why distribute storage?

One database node has finite storage, CPU, memory, disk I/O and network capacity, and is a failure domain. At larger scale we introduce replication, partitioning, or both.

2. Replication

Replication stores copies of the same logical data on multiple nodes.

```
Leader -> Replica A [same data]
-> Replica B [same data]
```

Benefits include availability, redundancy, disaster tolerance, geographically closer reads and read scaling. Replication alone does not automatically scale writes if all writes still pass through one leader.

3. Leader / follower

A common model sends writes to a leader while followers copy its changes. Reads may go to the leader or replicas depending on freshness requirements.

```
WRITE -> Leader -> Replica 1
-> Replica 2

READ -> Leader or replica
```

4. Replication lag

With asynchronous replication, followers can temporarily be behind the leader.

```
Leader: X=100
Replica: X=90 <- temporarily stale
later -> X=100
```

> **A read immediately after a successful write may return stale data if it goes to a lagging replica.**

5. Synchronous vs asynchronous replication

Synchronous: wait for replica acknowledgement before success; stronger durability/freshness trade-off but higher latency/coupling.

Asynchronous: leader can acknowledge first; lower latency but replicas lag and some failure modes can lose recently acknowledged data.

6. Failover

If a leader fails, an eligible follower may be promoted. Important questions are failure detection, selecting the most up-to-date replica, missing writes and redirecting clients. Deep consensus algorithms are outside Level 1.

```
Leader A [FAILED]
Follower B -> promoted leader
Follower C -> follows B
```

7. Read replicas

Read-heavy workloads can distribute eligible reads across replicas, but stale replicas are unsuitable for operations requiring the newest state.

8. Partitioning / sharding

Partitioning divides a dataset so different nodes own different subsets.

```
Users 0..999 -> Shard A
Users 1000..1999 -> Shard B
Users 2000..2999 -> Shard C
```

It increases total storage capacity and can distribute reads/writes, while introducing routing, rebalancing, hotspots and cross-shard operations.

9. Horizontal vs vertical partitioning

Horizontal: different rows/entities live on different partitions. This is what HLD interviews commonly mean by sharding.

Vertical: different columns/tables/features are separated.

10. Shard key

The shard key determines data placement. A good key distributes data and traffic while aligning with important access patterns.

```
shard = hash(device_id) % N
```

A key being unique does not make it a good shard key. Ask whether it distributes actual workload.

11. Range partitioning

Ranges make range queries natural, but skewed ranges can become hot. Time ranges are common for time-series data, but the newest range can receive almost all current writes.

12. Hash partitioning

Hashing usually spreads keys more evenly. However, naive modulo hashing can remap many keys when shard count changes, and cross-key range queries may become harder.

13. Hot partitions / hot keys

```
Shard A: 10k req/s
Shard B: 11k req/s
Shard C: 150k req/s <- HOT
```

Causes include celebrity/hot keys, very large tenants and time-based current-write partitions. Mitigations include better keys, salting, splitting hot partitions, caching hot reads or isolating large tenants.

14. Rebalancing

Adding/removing shards changes ownership. Moving large datasets while serving traffic consumes network, disk and CPU and requires routing updates.

15. Consistent hashing

Consistent hashing is designed to reduce the fraction of keys that move when nodes are added or removed compared with naive hash(key) % N.

```
hash(key)%3 -> add node -> hash(key)%4 -> many keys remap

consistent hashing -> smaller keyspace movement
```

Consistent hashing is about placement/remapping. It is not the same thing as data consistency.

16. Replication + sharding

```
Shard 1: Leader + replicas
Shard 2: Leader + replicas
Shard 3: Leader + replicas
```

Partitioning answers which nodes own which data. Replication answers how many copies of that data exist.

17. Stronger consistency

Rather than relying on vague labels, state the required behavior. Authoritative balances, executions, inventory reservations and security-sensitive state often need stronger guarantees and carefully controlled ordering.

18. Eventual consistency

Copies may temporarily disagree but converge after updates propagate.

```
Replica A: likes=1005
Replica B: likes=1003
later -> converge
```

This may be acceptable for social counters, analytics, caches, presence and other data where temporary staleness is tolerable.

19. Read-after-write consistency

After a successful update, the user's next read should reflect that update.

```
write profile=Alice -> immediate read -> Alice
```

Blindly routing that immediate read to a lagging asynchronous replica can violate this requirement.

20. Monotonic reads

A client should not see version 10 and then version 8 because subsequent reads went to replicas at different lag positions. Consistent replica selection or version-aware routing can help.

21. Consistency vs availability during failures

Network/node failures can force trade-offs: reject or delay operations to preserve stronger correctness, or continue serving with potentially stale/divergent data when availability is more important.

> **Do not turn every interview into a CAP lecture. Explain the concrete failure and application requirement.**

22. Cross-shard operations

```
Account A -> Shard 1
Account B -> Shard 7
Transfer -> coordination across shards
```

Transactions, joins and aggregates spanning shards are more expensive and complex. A good shard key tries to colocate frequently related operations where practical.

23. Secondary access patterns

If data is sharded by user_id but queried frequently by email, the system needs a way to locate the owning shard, such as an additional distributed/global index or lookup mapping.

24. C++ service considerations

C++ services may need endpoint-aware connection pools, shard routing tables, bounded pending queues, timeout budgets, retry/idempotency logic and careful buffer/result ownership.

```
Request -> shard router -> connection pool -> shard leader/replica
```

Blind retries after ambiguous write failures can duplicate operations. Reliability and idempotency are covered later.

25. Camera telemetry example

Millions of cameras can be partitioned using a well-distributed device identifier, with time-oriented organization within each partition. Replicas provide redundancy. Tenant ID alone may be poor if one customer generates most

traffic.

26. Replication vs partitioning summary

Replication: same data on multiple nodes -> redundancy, availability, read scale.

Partitioning: different subsets on different nodes -> total storage and distributed read/write load.

Both: each partition has multiple copies.

27. Interview checklist

1. Is one node insufficient for storage, throughput or availability?
2. Need more read capacity? Consider replicas.
3. Need more storage/write capacity? Consider partitioning.
4. What shard key?
5. Can it hotspot?
6. Which operations become cross-shard?
7. Sync or async replication?
8. Can reads tolerate lag?
9. What happens on leader failure?
10. How is rebalancing handled?
11. What consistency behavior is actually required?

28. Common mistakes

Replication = sharding: false.
Replicas automatically scale writes: not necessarily.
More shards always help: false; coordination complexity rises.
Unique key = good shard key: false.
Async replica is always current: false.
Hashing eliminates hotspots: a hot individual key can remain hot.
Consistent hashing means data consistency: false.
Every read needs strongest consistency: usually unnecessary.

29. Practice scenarios

1. Product DB is 95% reads and the leader is overloaded by reads. What could you add, and what new problem appears?

2. User changes profile name, then immediately reads from a lagging replica and sees the old name. What requirement was violated?

3. One DB stores 5 TB but dataset will grow to 40 TB. What concept becomes important?

4. You shard by tenant_id, but one tenant generates 60% of traffic. What problem occurs?

5. Define replication and partitioning in one sentence each.

6. Why can hash(key)%N cause large remapping when N changes, and what technique reduces this?

7. Would eventual consistency be acceptable for a like count? What about an authoritative bank balance?

30. 60-second revision

Replication: copy same data.
Sharding: split different data.
Leader: commonly accepts writes.
Replica lag: follower can be stale.
Sync replication: more acknowledgement/coupling.
Async: lower latency, lag risk.
Shard key: determines placement.
Hotspot: disproportionate load.
Consistent hashing: reduce remapping on membership changes.
Read-after-write: see your successful update.
Eventual: temporary disagreement allowed; later convergence.

31. Ready for Level 1.7 when...

You can distinguish replication from partitioning, explain replica lag/failover, critique shard keys and hotspots, explain consistent hashing's purpose, identify cross-shard costs, and choose consistency behavior from requirements.